김진우

KIM JINWOO

게임 클라이언트 프로그래머

플레이하며 찾은 문제를
구조로 해결합니다.

한눈에 보기

01 **출시·운영까지 경험**
전기톱 키우기 · 토스 앱 미니게임
공모전 출품 → 출시 → 업데이트·오류 대응

02 **팀에서 가장 많은 코드**
Knight Only · Indie Go
플레이어 코드 약 79% · 팀 최다(약 44%)

03 **엔진 없이 만든 게임 루프**
MapleLikeClient · C++ · SFML
충돌 · 입력 보정 · 데이터 레벨

TECH STACK
Unity · C# / C++ · SFML

CONTACT
jinu217@naver.com

GITHUB · SITE
github.com/jinu217 >
jinu217.github.io >

OVERVIEW

프로젝트 한눈에 보기

2025.08부터 지금까지 만든 7개 프로젝트입니다. 줄을 누르면 해당 페이지로 이동합니다.

기간	프로젝트	팀 · 역할	핵심	쪽
2025.12~진행 중	**Knight Only** 2D 액션 로그라이크 졸업작품	4인 · 프로그래머	플레이어·전투·BPM·적/보스 AI·씬 전환	04
2025.10~11 · 운영	**전기톱 키우기** 토스 앱 미니게임 출시	7인 · 프로그래머	과열 규칙 보정, 진동 연동, 출시 뒤 업데이트	13
2026.04~진행 중	**Indie Go** 윷놀이 (온라인·로컬)	8인 · PM / 프로그래머	팀 최다 코드, 협업 구조, 윷 던지기	18
2026.07~08	**MapleLikeClient** C++ · SFML 2D 액션	개인	게임 루프·충돌·입력 보정·데이터 레벨	24
2026.08 · 24시간	**Wheel of Fortune** 교내 동아리 게임잼	4인 · 단독 프로그래머	약 17시간에 MVP, Windows 배포	28
2025.08~11	**컴공과 키우기** 교내 동아리 · Android	7인 · PM / 프로그래머	보스 5종, 학교 축제 시연	29
2025.08~11	**Tishoo** 기업 연계 수업 과제 · 웹	7인 · 단독 개발자	화면·서버·DB·배포 (게임 외 경험)	30

읽는 법 모든 페이지 아래에 근거(커밋, 소스 파일, 코드 작성 비율)를 적었습니다. 팀원과 함께 만든 부분은 "함께"로 구분했습니다.

HIGHLIGHTS

대표 문제 해결 3선

직접 플레이하고 테스트하며 찾은 문제를 원인부터 고친 사례입니다. 카드를 누르면 상세 페이지로 이동합니다.

01

연타로 과열을 피하던 허점

전기톱 키우기

문제 짧게 끊어 누르면 열은 거의 안 쌓이는데 피해는 계속 들어갔습니다.

해결 수치를 비교·분석하고 기획과 협의해 퀵탭 보정을 넣었습니다.

결과 연타에도 과열이 쌓입니다. 출시 뒤 강화 수치도 다시 맞췄습니다.

자세히 보기 14쪽 >

02

연타 때 빠지던 진동

전기톱 키우기

문제 300ms 안의 진동 요청을 버려, 실기기에서 연타 시 진동이 빠졌습니다.

해결 요청을 큐에 쌓아 120ms 간격으로 차례로 재생하게 바꿨습니다.

결과 연속 벌목에도 진동 누락이 줄었습니다.

자세히 보기 15쪽 >

03

대시·넉백 중 어색한 움직임

Knight Only

문제 대시·넉백 중에 일반 이동이 속도를 덮어썼습니다.

해결 행동 상태의 우선순위를 정하고, 대시 종료 시 중력을 복구했습니다.

결과 대시·넉백이 의도한 궤적대로 움직입니다.

자세히 보기 05쪽 >

근거 세 사례 모두 커밋 기록과 코드로 확인할 수 있습니다: 퀵탭 보정 dd42cf6, 진동 큐 f2ea160, 행동 상태 조건 PlayerController.cs.

Knight Only · 대표작

2025.12 기획 · 2026.04 개발 시작 ~ 진행 중 | 4인 팀(프로그래머 3 · 디자이너 1) · 프로그래머 | Unity · C# · New Input System | 2D 액션 로그라이크 졸업작품



내 담당	플레이어 이동·전투, BPM/SP 시스템, 적·보스 AI(Stage 1~5), 인벤토리·장비, 튜토리얼, NPC 상점, 씬 전환
기여 근거	팀 저장소 커밋 423개 중 113개. PlayerController 약 79%, EnemyController 약 76%, 튜토리얼·씬 전환 관리자 전부 작성
진행	2026.10~11에도 이단 점프, 스킬 이펙트, 처치 수, 사운드 작업을 이어 가고 있습니다





근거 코드 작성 비율은 팀 저장소(merge/1104 브랜치, 2026-11-09)의 줄 단위 기록 기준입니다. 공개 저장소는 백업본입니다.

Knight-Only_Backup / GitHub >

대시·넉백 중 어색해지던 움직임

대시나 넉백 중에도 일반 이동이 속도를 덮어쓰던 문제를 행동 상태의 우선순위로 고쳤습니다.

상황 대시나 넉백 중에 일반 이동 코드가 속도를 덮어써, 움직임이 의도한 궤적에서 벗어나 어색해졌습니다.

발견 직접 플레이하다 발견했습니다. 입력과 상태가 바뀌는 순간의 코드를 따라가며 원인을 찾았습니다.

원인 일반 이동 처리가 현재 행동 상태를 확인하지 않고 매 프레임 속도를 다시 설정했습니다.

해결 행동 상태의 우선순위를 정해 대시·넉백·이동 지연 중에는 일반 이동을 건너뛰게 했습니다. 대시가 끝나면 보관한 중력 배율과 상태를 되돌립니다.

결과 대시와 넉백이 끝까지 의도한 궤적으로 움직입니다. 같은 원칙으로 공격·인벤토리 입력도 상태에 맞게 막았습니다.

행동 상태에 따른 입력 차단

```
795  if (isDashing || isKnockback || isMoveDelayed) return;
     ...
808  if (isDashing || isKnockback || isMoveDelayed ||
         isInventoryOpen || ChestRewardPanel.IsAnyOpen)
         return;
     ...
1102 if (!context.started || attackMovementLocked ||
         isDashing || isKnockback || isInventoryOpen ||
         ChestRewardPanel.IsAnyOpen) return;
```

근거 이동을 건너뛰는 조건은 본인의 첫 커밋(2026-04-26)부터 들어 있고, 지금 PlayerController.cs의 약 79%가 본인 작성입니다.

PlayerController.cs / Knight-Only_Backup >

공격 한 번에 여러 번 들어가던 피해

Collider가 여러 개인 적에게 피해가 중복되던 문제를, 충돌체가 아닌 피해 대상의 ID로 판정해 막았습니다.

상황 적과 보스에 Collider가 여러 개 붙어 있어, 공격 한 번에 같은 대상이 여러 번 피해를 받았습니다.

원인 물리 질의는 Collider 단위로 결과를 돌려주는데, 피해도 Collider마다 전달했습니다.

해결 Collider마다 상위 IDamageable을 찾아 실제 피해 대상의 ID를 HashSet에 넣고, Add가 false면 건너뜁니다. 집합은 판정마다 새로 만듭니다.

결과 공격 한 번에 대상 하나가 한 번만 맞습니다. 일반 적과 보스가 같은 IDamageable 계약으로 피해를 받습니다.

대상 ID 집합으로 중복 판정

```
1371  HashSet<int> damagedIds = new HashSet<int>();
      ...
1389  IDamageable target =
          hit.GetComponentInParent<IDamageable>();
1390  if (target == null) continue;
      ...
1397  if (targetBehaviour == null) continue;
1398  int id = (target as UnityEngine.Object)?.GetInstanceID()
          ?? 0;
1399  if (!damagedIds.Add(id)) continue;
```

HashSet.Add	평균 O(1), 넣으면서 중복 여부를 바로 확인
List 검색	매번 O(u), m개 판정이면 O(m×u)

근거 피해 대상 ID 집합(damagedIds)은 본인 커밋 "fix: Enemy knockback system"(2026-05-27)에서 처음 들어갔습니다.

PlayerController.cs / Knight-Only_Backup >

BPM 배율이 겹쳐 쌓이던 구조

BPM 구간이 바뀔 때마다 기본 스탯을 고치던 방식을, 행동 시점에 최종 값을 계산하는 방식으로 바꿨습니다.

상황 BPM은 공격·대시로 오르는 전투 상태값입니다. 구간에 따라 공격력·공격속도·이동속도·대시·SP 수급에 배율이 붙고, BPM 보너스가 중복 계산되는 문제가 있었습니다.

원인 BPM이 바뀔 때마다 기본 스탯 자체를 바꾸면 배율이 겹치거나 원래 값으로 정확히 돌아오지 않습니다.

해결 기본값은 그대로 두고, 행동이 일어나는 시점에 현재 구간의 배율을 곱해 최종 값을 계산하도록 구조를 바꿨습니다.

결과 구간이 오가도 값이 쌓이지 않습니다. 탈진, 자연 감소, 명상, SP 수급까지 같은 구조로 연결했습니다.

A · 130 이하	기본 배율
B · 130~170	공격력·대시 속도·거리 배율
C · 170~200	B + 공격·이동 속도 배율
D · 탈진	200 초과 시 진입, 이동 정지·공격 제한

구간별 배율은 읽을 때 계산

```
141  public float AttackDamageMultiplier
142      => CurrentBpmRank == BpmRank.B || CurrentBpmRank ==
             BpmRank.C ? tensionAttackDamageMultiplier : 1f;
144  public float AttackSpeedMultiplier
145      => CurrentBpmRank == BpmRank.C ?
             feverAttackSpeedMultiplier : 1f;
```

근거 PlayerStatus는 팀원이 처음 만든 클래스에 본인이 BPM·SP·명상을 더했습니다(2026-06-25). 지금 코드의 약 58%, BPM 관련 줄 대부분이 본인 작성입니다.

PlayerStatus.cs / Knight-Only_Backup >

적·보스 AI와 Stage 1~5 보스

공통 적 컨트롤러에서 근거리·원거리 적을 나누고, 보스는 공통 동작과 스테이지별 패턴을 분리했습니다.



Stage 2 메인 보스 등장 장면 (2026-11 캡처)

공통 행동과 개별 공격

탐지 범위·시야·공격 거리·공격 간격과 피격 반응은 EnemyController가 공통으로 처리하고, 적마다 공격 방식만 다르게 구현합니다. 새 적을 더할 때 반복 구현이 줄어듭니다.

보스 스킬 선택

스킬 목록을 앞에서부터 검사해 조건을 만족하는 첫 스킬을 고릅니다. 목록 순서가 우선순위이고, 실행 중에는 새 스킬을 고르지 않습니다.

탐지 → 반응 → 추격 → 공격

근거 EnemyController 약 76%, MidBossController 약 61%가 본인 작성이고, BossSkillExecutor는 팀원과 거의 반씩 작성했습니다. Stage 2~5 보스는 본인 커밋(2026-09~10)입니다.

EnemyController.cs / Knight-Only_Backup >

Stage 2 보스의 욕구 스킬 · 안전 필드

보스가 필드를 깔고 그 안에서 버티는 전용 패턴입니다. 효과·복귀 규칙·실행 순서를 한 컴포넌트에 모아 관리했습니다.



Stage 2 메인 보스 등장 장면 (2026-11 캡처)

발동	페이즈 3 이상 · 쿨타임 60초 · 필드 25초
방어	받는 피해 25% 감소, 필드 안에서는 75%
강화	공격력 1.1배 · 3초마다 최대 체력 3% 회복
반동	필드가 끝난 뒤 30초 동안 받는 피해 25% 증가

필드 밖 규칙

보스가 필드 밖에 5초 있으면 복귀 공격을 하고, 10초가 되면 강제로 돌아갑니다. 강제 복귀는 일반 연격과 복귀 공격·스턴보다 우선합니다. 필드 밖에서는 플레이어가 들어오거나 보스를 때린 뒤에만 전투를 이어 갑니다.

실행 순서 처리

필드 종료와 회복은 보스가 공격하는 동안에도 Update에서 갱신합니다. 보스 AI가 컴포넌트 Update보다 먼저 호출되는 프레임에도 만료된 필드를 쓰지 않도록, AI 쪽 Tick에서도 한 번 더 확인합니다.

만료 확인 · 강제 복귀 우선

```
117  if (fieldActive && (fieldVisual == null || Time.time >= fieldEndTime))
         RemoveField();
     ...
135  if (outsideTime >= forcedReturnDelay)
```

근거 Stage2_MainBossDesireSkill.cs 443줄 중 362줄(약 82%)이 본인 작성입니다. 처음 커밋은 "feat: Stage2_MainBoss DesireSkill"(2026-09-15)입니다.

Knight-Only_Backup / GitHub >

흩어진 씬 이동을 하나로 모은 전환 관리자

여러 스크립트에 흩어져 있던 씬 이동을 비동기 전환 관리자로 모으고, 진입 위치와 저장 시점을 한곳에서 다룹니다.

이동 요청 → 로딩 패널 → 전환 전 저장 → 비동기 로드 → Spawn ID 위치

하나의 흐름

SceneTransitionManager가 로딩 패널을 띄우고, 전환 전에 저장한 뒤 LoadSceneAsync로 다음 씬을 불러옵니다. 진행 연출이 끝나야 씬을 활성화합니다.

진입 위치와 저장

Spawn ID로 다음 씬의 진입 위치를 정합니다. 저장은 팀원이 나눈 영구·회차 데이터 구조(PersistentData·RunData)를 함께 확장해 마을·상점·스테이지 진행을 담았습니다.

로딩 → 저장 → 비동기 로드

```
139  SaveBeforeTransition(player, transition);
141  AsyncOperation loadOperation =
         SceneManager.LoadSceneAsync(transition.TargetSceneName);
     ...
148  loadOperation.allowSceneActivation = false;
     ...
190  SaveData.Instance.run.spawnId = transition.TargetSpawnId;
```

근거 SceneTransitionManager는 2026-08-27 "feat: Loading" 커밋에서 본인이 만들었고 모든 줄이 본인 작성입니다. 저장 구조 분리는 팀원이 먼저 만들었습니다(2026-07-14).

SceneTransitionManager.cs / Knight-Only_Backup >

그 밖에 맡은 시스템

상점·장비·발판·튜토리얼처럼 플레이 흐름을 받치는 시스템도 직접 구현했습니다.



NPC 상점

장비·대장간·버프 상점을 NPCShopBase에서 공통 처리하고, 상점마다 다른 부분만 구현했습니다(약 94% 본인 작성).



장비 재계산

장비를 바꾸면 기본 스탯에서 다시 계산해 효과가 거듭 쌓이지 않게 했습니다. 기본 구조는 팀원과 함께, 재계산 함수는 본인이 다시 작성했습니다.



하향 점프

시작 발판과의 충돌만 잠시 끄고, 몸체가 발판 아래로 충분히 내려가면 다시 켭니다.

enum TutorialStep (16)

```
0  BasicControls
4  EmpoweredDownJumpUnlocked
6  DashUnlocked
9  InventoryEquipGuide
11 FirstDummyAttackable
15 GoTown
```

튜토리얼 단계

16단계(TutorialStep)로 나누고, 현재 단계가 예상 단계와 같을 때만 다음으로 넘어갑니다. 대시·공격·인벤토리는 단계에 따라 허용합니다. TutorialManager는 모두 본인 작성입니다.

근거 모두 팀 저장소의 본인 커밋으로 확인한 내용입니다. 상점 화면은 2026-11 캡처, 장비·발판 화면은 기존 포트폴리오 캡처입니다. 단계 이름은 실제 enum에서 일부를 옮겼습니다.

Knight-Only_Backup / GitHub >

팀이 함께 읽기 좋은 코드로 정리

개발 초기에 기능 추가와 별도로, 팀원들이 함께 읽고 고칠 핵심 스크립트를 정리하는 커밋을 남겼습니다.

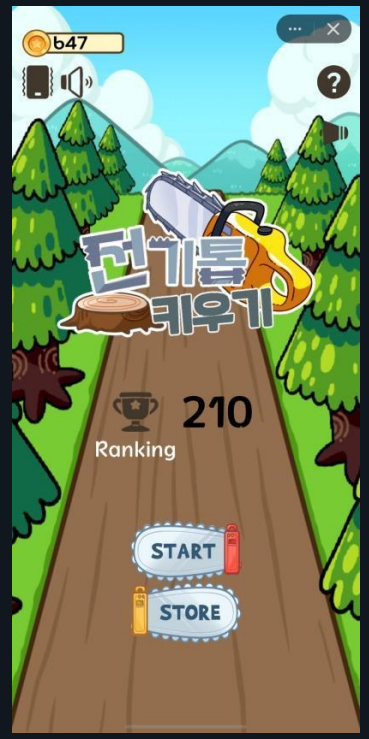
한국어 주석	플레이어·적·원거리 적·투사체·적 데이터·게임 매니저 등 핵심 스크립트 7개에 한국어 주석을 붙였습니다(7개 파일, +301 / −629줄).	3104419e · 2026-04-30
흐름 정리	적의 반응 흐름을 고치고, 중복된 공격 타이머를 없애고, GameManager가 없을 때를 막는 null 방어를 넣었습니다.	f4dd4dd9 · 2026-04-27
API 교체	폐기 예정인 FindObjectOfType를 FindFirstObjectByType로 바꿨습니다.	569d8604 · 2026-04-27
수식 정리	포션 회복량 계산을 "기본값 + 기본값 × 비율"로 바꿨습니다.	a335d043 · 2026-04-27
버전 정리	팀 프로젝트의 Unity 버전을 6000.3.15f1로 올리고 설정을 맞췄습니다.	da30d797 · 2026-05-09

근거 모두 팀 저장소의 본인 커밋입니다. 본인의 첫 커밋(2026-04-26) 직후에 이런 정리 커밋을 먼저 남겼습니다.

Knight-Only_Backup / GitHub >

전기톱 키우기 · 출시와 운영

2025.10~11 개발 · 공모전 기간 마지막 날 출시 · 이후 운영 | 7인 팀(기획 1 · 프로그래머 4 · 디자이너 2) · 프로그래머 | Unity · C# · WebGL · 앱인토스







내 담당	전기톱 이동·공격·과열·반동과 성장 수치, 나무·바위·몬스터, 보스 구간, 앱인토스 진동 연동, WebGL 빌드·앱인토스 적용 일부
결과	앱인토스 공모전에 출품해 토스 앱 미니게임으로 출시했고 지금도 서비스 중입니다. 광고 수익이 작게 발생했습니다
운영	출시 뒤에도 업데이트와 오류 대응을 맡았습니다

11.17 출시 빌드 → 11.30 보스 3·4·5 추가 → 12.01 수치·시작 UI 수정

근거 출시와 업데이트 시점은 앱인토스 저장소의 본인 커밋("last build", "feat: Boss 3/4/5", "fix: attackspeed/overhit") 기준입니다.

app-in-toss / game 브랜치 >

연타로 과열을 피하던 허점

기획 수치대로 구현했지만 입력 방식과 피해 판정 시점이 맞물려 생긴 허점을, 직접 찾아 기획과 함께 막았습니다.

상황 과열 수치는 기획이 준 값 그대로였습니다. 그런데 아주 짧게 눌렀다 떼기를 반복하면 열은 거의 쌓이지 않고 피해는 계속 들어갔습니다.

발견 직접 테스트하다 발견했습니다. 누름 시간 대비 오르는 과열과 뗐을 때 떨어지는 과열을 비교하고, AI로 연타가 통하는 구간을 수치로 뽑아 직접 플레이로 확인했습니다.

원인 순간적인 누름은 열이 거의 차지 않고 뗀 사이에 식기까지 했습니다. 피해는 누르는 즉시 들어가 연타가 이득이었습니다.

해결 기획과 협의해 0.5초 안에 뗀 퀵탭은 누적을 2배로 하고 최소치를 더했습니다. 뗀 뒤 0.2초는 식지 않게 하고 누름 길이에 따라 냉각 속도를 나눴습니다.

결과 연타해도 과열이 쌓입니다. 출시 뒤에는 과열 강화 보너스를 고정값 더하기에서 기본 한계의 비율로 바꿨습니다.

퀵탭 보정과 냉각 속도 분리

```
290 if (holdElapsed <= quickTapWindow)
292     float baseAdded = overhitRate * holdElapsed;
293     float desiredTotal = baseAdded * Mathf.Max(1f,
            quickTapMultiplier);
294     float extra = Mathf.Max(0f, desiredTotal -
            baseAdded);
296     overhit += extra + quickTapFlatBonus;
    ...
315 if (timeSinceRelease > quickTapcoolDelay)
317     if (lastHoldDuration > quickTapWindow)
319         overhit -= Time.deltaTime * coolRate;
321     else
323         overhit -= Time.deltaTime * quickTapcoolRate;
```

근거 퀵탭 보정 dd42cf6(2025-10-24, 1차 빌드 다음 날) → 게이지 수정 363a09d 외 2개(10-27~28) → 출시 뒤 비율 조정 d4a2bfa(12-01). 모두 본인 커밋입니다.

dd42cf6 퀵탭 보정 > d4a2bfa 출시 뒤 조정 >

Unity와 토스 앱을 잇는 진동, 연타 때의 누락

Unity WebGL에서 앱인토스 진동 기능을 부르는 연결 코드를 만들고, 연타 때 진동이 빠지던 처리를 큐 방식으로 고쳤습니다.

상황 나무가 쓰러질 때 토스 앱의 진동을 울려야 했지만, Unity WebGL 코드는 앱인토스 진동 API를 바로 부를 수 없었습니다.

연결 C# Haptics.Generate가 WebGL 템플릿의 JavaScript 함수를 부르고, 그 함수가 앱인토스 진동 API(없으면 브라우저 진동)를 호출하게 했습니다.

발견 처음 처리는 300ms 안에 들어온 요청을 버려, 연달아 나무를 쓰러뜨리면 진동이 빠졌습니다. 실제 휴대폰에서 확인했습니다.

해결 요청을 버리지 않고 큐(최대 20개)에 쌓아 120ms 간격으로 차례로 재생하고, 앱이 가려지면 재생을 멈추게 했습니다.

결과 연속 벌목에도 진동 누락이 줄었습니다. 가까운 요청을 합치는 방식도 설정으로 남겼습니다.

처음: 300ms 안의 요청은 버림

```
15  let last=0, GAP=300, ONCE=60;
    ...
17  window.AppsInToss_generateHaptic = async
        function(type){
    ...
19  if(!throttle()) return;
```

수정: 큐에 쌓아 차례로 재생

```
19  const MAX_QUEUE  = 20;
    ...
69  window.AppsInToss_generateHaptic = function(type){
70    if (MODE === 'coalesce') {
71      if (Date.now() - lastAt < GAP_MS) return;
72    } else {
73      if (q < MAX_QUEUE) q++;
75    pump(type);
```

근거 진동 연결 97661ff → 큐 처리 f2ea160(같은 날 2025-11-01). 두 커밋 모두 본인 작성이고, 진동 누락은 실제 휴대폰에서 확인했습니다.

97661ff 진동 연결 >

f2ea160 큐 처리 >

1차 빌드 직후 찾은 실제 환경 문제 두 가지

빌드를 실제로 돌려 보며 화면 겹침과 오브젝트 반환 문제를 이틀 안에 고쳤습니다.

게임오버와 일시정지 패널 겹침

2025-10-22 · a3d87c2

문제 게임오버 화면이 뜰 때 일시정지 패널이 겹쳐 보였습니다.

해결 게임오버가 뜨는 순간 일시정지 패널을 숨기고 일시정지 버튼을 비활성화했습니다.

게임오버 시 일시정지 정리

```
137  if (pm.pausePanel) pm.pausePanel.SetActive(false);
138  if (pm.pauseButton) pm.pauseButton.interactable = false;
```

게임이 멈춘 상태의 나무 반환

2025-10-23 · 67624fc

문제 쓰러진 나무를 치우는 처리가 게임 시간에 묶여, 게임이 멈춘 상태에서는 사라지지 않을 수 있었습니다.

해결 실제 시간(WaitForSecondsRealtime) 기준으로 비활성화해 풀로 돌려보냈습니다. 재사용 초기화는 팀원과 함께 설계했습니다.

실제 시간 기준 비활성화

```
155  StartCoroutine(DeactivateAfterRealtime(duration + 0.15f));
     ...
160  yield return new WaitForSecondsRealtime(sec);
161  if (gameObject != null) gameObject.SetActive(false);
```

근거 두 수정 모두 본인 커밋입니다. 코드 주석에 "게임 멈춰도 확실히 사라짐"이라고 의도를 남겼습니다.

a3d87c2 패널 겹침 > 67624fc 나무 반환 >

전기톱 상태 · 성장 수치 · 보스 구간

입력 유지 → 벌목 → 보상 → 강화로 이어지는 흐름을 만들고, 강화 결과가 조작감에 바로 반영되게 했습니다.



입력 유지 → 벌목 → 보상 → 강화 → 다음 구간

입력·목표·과열 상태

누르는 시간만큼 열을 쌓고 한계에 닿으면 과열로 바꿉니다. 가까운 나무가 바뀌면 UI 대상과 첫 타격을 갱신하고, 목표가 없거나 손을 떼면 공격 타이머를 초기화합니다.

성장 수치

강화 결과를 공격력·공격 간격·과열 한계·반동에 반영하고 값의 범위를 제한합니다. 강화할수록 진행 속도와 조작감이 달라집니다.

보스 구간

보스 전용 구간에는 일반 나무·바위를 만들지 않도록 생성기에 보스 구간 집합을 두고, 보스는 앞뒤 나무 사이 열 가운데에 배치했습니다.

역할별 관리자

게임 진행 상태·코인·타이머·게임오버·상점을 역할별 Manager로 나누고 전기톱·오브젝트 시스템과 연결했습니다.

근거 보스 구간 처리는 본인 커밋 4431974(2025-11-16)입니다. 전기톱 코드(ChainSaw_Move.cs)는 본인이 처음 만들었고 지금 코드의 약 84%가 본인 작성입니다.

ChainSaw_Move.cs / app-in-toss >

Indie Go · 윷 아레나

2026.04 ~ 진행 중 | 8인 팀(기획 2 · 개발 4 · 디자인 2) · PM 겸 프로그래머 | Unity · C# · Input System | 윷놀이 (온라인 · 로컬)



내 담당	초기 프로젝트·폴더 구조와 공통 정의, UI 전반(시작·로비·캐릭터 선택·인게임·종료), 윷 던지기, 오디오·그래픽·접근성 설정, PM
기여 근거	스크립트 약 1만 4,600줄 중 약 6,500줄(약 44%) 작성, 커밋 74개로 팀에서 가장 많습니다
진행	창업 동아리에서 개발 중이며, 학교 창업 캠프에서 MVP를 발표·시연했습니다



근거 코드 비율은 로컬 저장소 develop 브랜치(2026-11-07 커밋) 기준입니다. 시작 화면은 2026-11 캡처입니다.

HK-INDIE-GO / GitHub >

서로 기다리지 않고 동시에 개발하는 구조

PM으로서 작업량을 똑같이 나누기보다, 한 사람의 작업을 기다리느라 다른 사람이 멈추는 상황을 줄이는 데 중점을 뒀습니다.

상황 개발자 4명이 동시에 작업하면 서로의 완료를 기다리거나 수정 범위가 겹칠 수 있었습니다.

공통 개발 초기에 Common 영역을 나누고 보드·규칙·플레이어·턴·윷 결과·시작 설정을 공통 정의 6개로 만들었습니다(2026-07-04).

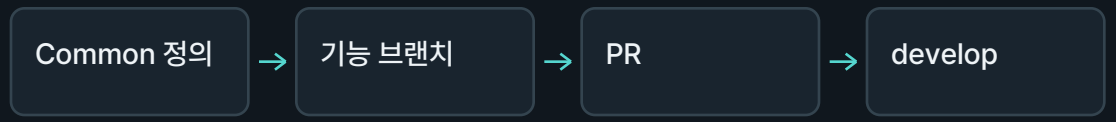
분담 기능 간 선후 관계와 의존성을 먼저 정리해 각자 독립적으로 구현하고, develop 기준 기능 브랜치와 PR로 정해진 지점에서 합쳤습니다.

문서 작업 경계를 문서로 넘겼습니다. 팀전 모드에서는 값의 전달 경로와 사용 예시를 남기고 "팀전별 규칙은 인게임 담당 범위"라고 표시했습니다.

결과 여러 개발자가 수정 범위가 겹치지 않게 동시에 작업했고, 창업 캠프 발표용 MVP를 함께 완성했습니다.

TeamModeHandoff.md (2026-11-03)

```
# 팀전 모드 전달
Team1Btn → TeamMode1,  Team2Btn → TeamMode2
로비 RoomSettingsData.teamMode
  → GameStartSettings.teamMode
인게임: GameStartSettingsHolder.Current 로 접근
개인전에서는 teamMode 값을 쓰지 않음
팀전별 게임 규칙 구현은 인게임 담당 작업 범위
```



근거 공통 정의 6개(8ed3af3)와 TeamModeHandoff.md(c7348ba)는 본인 커밋입니다. 문서 카드는 원문을 줄여 옮겼습니다.

8ed3af3 공통 정의 >

c7348ba 팀전 모드 전달 >

결과를 먼저 정하는 윷 던지기

던진 결과가 게임 진행을 정하므로, 물리 엔진에 맡기지 않고 결과를 먼저 정한 뒤 그 결과대로 떨어지는 연출을 계산합니다.



왜 물리를 쓰지 않았나

Rigidbody 중력이나 투명 벽에 맡기면 판 밖으로 튀거나 결과가 애매해질 수 있습니다. 결과를 먼저 정하고, 윷이 항상 원판 안에 결과대로 착지하도록 궤적을 계산했습니다.

결과 면과 착지 순서

필요한 윷만 180도 돌리고 마지막 윷을 백도 윷으로 둡니다. 착지 순서는 네 개 기준 8가지 묶음(1-2-3-4 ~ 1234) 중 무작위로 고릅니다.

결과 결정 → 면 배치 → 자세 계산 → 연출

근거 왼쪽은 "걸" 결과로 던진 장면입니다(2026-11 캡처). YutThrowManager 492줄은 모두 본인 작성이며, 처음 버전부터 물리 없이 착지시키는 설계였습니다.

YutThrowManager.cs / HK-INDIE-GO >

겹친 윷의 착지 자세 계산과 검증

윷이 서로 파고들거나 판을 뚫지 않도록 착지 자세를 계산하고, 그 결과를 반복 검증하는 스크립트를 만들었습니다.

착지 자세 계산

겹쳐 떨어지는 윷은 접촉하는 꼭짓점의 높이 조건을 모두 만족하는 가장 낮은 받침 자세를 찾습니다. 두께가 있는 모델의 기울어진 끝까지 포함한 보수적인 외곽으로 계산합니다.

역할 분리

한 파일에 있던 처리를 착지 계산(YutLandingSolver), 연출(YutThrowAnimator), 표시(YutVisual)로 나눴습니다(2026-10-31).

검증 스크립트

시드 100개로 판 표면 관통, 원판 밖 이탈, 윷끼리의 상하 겹침을 격자 표본으로 검사합니다. 플레이어 빌드에서는 빠지는 검증 코드입니다.

YutLandingChecks · 착지 기하 검증

```
18  for (int seed = 0; seed < 100; seed++)
20      UnityEngine.Random.InitState(seed);
21      bool longY = seed % 2 == 0;
22      Vector3 size = longY ? new Vector3(0.25f, 1.37f, 0.19f) :
            new Vector3(1.37f, 0.25f, 0.19f);
23      var poses = YutLandingSolver.Solve(new[] { size, size,
            size, size },
24          new[] { 0.705f, 0.705f, 0.705f, 0.705f }, 2f, 1.3f,
                0.65f, longY, true, 0.02f, 12f);
25      for (int i = 0; i < poses.Length; i++)
27          var p = poses[i];
28          if (float.IsNaN(p.Height)) throw new
                Exception("Invalid landing height");
30          if (p.Height - Mathf.Abs(p.Slope) * p.HalfLength -
                p.HalfThickness < -0.0001f)
31              throw new Exception("Board penetration");
```

근거 YutLandingSolver(123줄)와 검증 스크립트는 모두 본인 작성입니다. 분리와 검증 추가는 커밋 fbc303a(2026-10-31)입니다.

fbc303a 착지 계산·검증 >

플레이어를 위한 입력 · 화면 · 설정

여러 명이 한 화면에서 참가하는 로컬 입력과, 연출 속도·글자 크기 같은 접근성 설정을 공통 기능으로 만들었습니다.



설정 화면 일부: 연출 속도 · 접근성 · 글자 크기 · 해상도 (2026-11 캡처, 개발 중 화면)

로컬 참가와 캐릭터 선택

키보드·게임패드 참가를 플레이어 슬롯 단위로 관리하고, 슬롯마다 커서와 선택 완료 여부를 둡니다. 확정 중에는 중복 시작을 막고 결과를 다음 씬으로 넘깁니다.

접근성 설정

연출 속도(1배·1.5배)와 UI 글자 크기(3단계)를 고르고 저장합니다. 말 이동과 윷 던지기 연출이 이 속도를 따릅니다.

턴 이벤트 UI

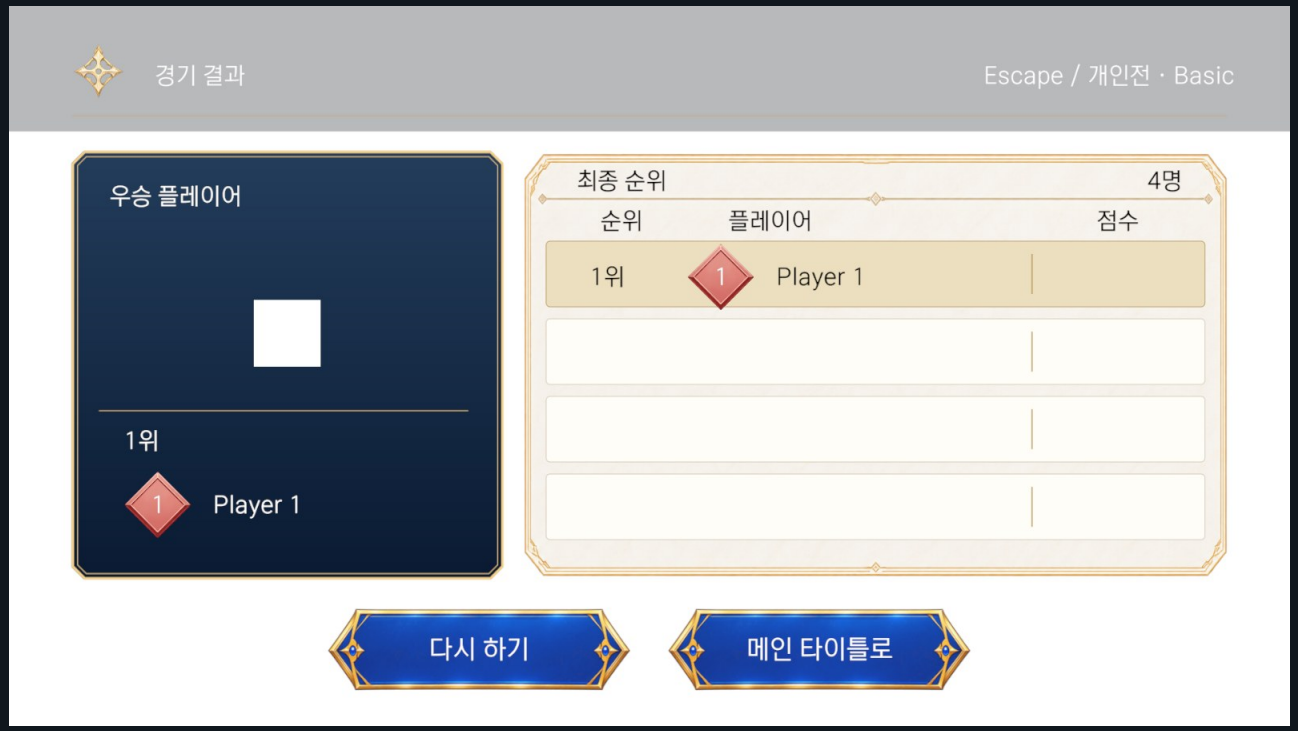
턴 시작·단계 변경 이벤트를 OnEnable에서 구독하고 OnDisable에서 해제해, 꺼진 UI가 이벤트를 받지 않게 했습니다.

근거 시작 화면·로비·캐릭터 선택·인게임 UI·게임 종료 화면과 설정 스크립트는 거의 모든 줄이 본인 작성입니다(예: 로비 UI 728줄 전부).

HK-INDIE-GO / GitHub >

게임 결과 화면과 팀전 모드

개인전과 팀전의 결과를 한 화면에서 보여 주고, 결과 데이터가 일부 빠져도 순위가 올바르게 보이도록 예외를 처리했습니다.



결과 데이터를 직접 넣어 띄운 결과 화면 (2026-11 캡처, 캐릭터 이미지 자리는 비어 있음)

개인전과 팀전

게임이 끝나면 결과 이벤트를 받아 개인전은 우승 플레이어와 순위를, 팀전은 우승 팀 구성원을 채웁니다. 이벤트는 OnEnable에서 구독하고 OnDisable에서 해제합니다.

빠진 데이터 처리

2팀 대전에서 순위 데이터가 생략되면 우승 팀 외 참가 팀을 2위로 표시합니다. 우승 팀 구성원은 공유 캐릭터 번호가 아니라 실제 참가자 번호로 찾습니다.

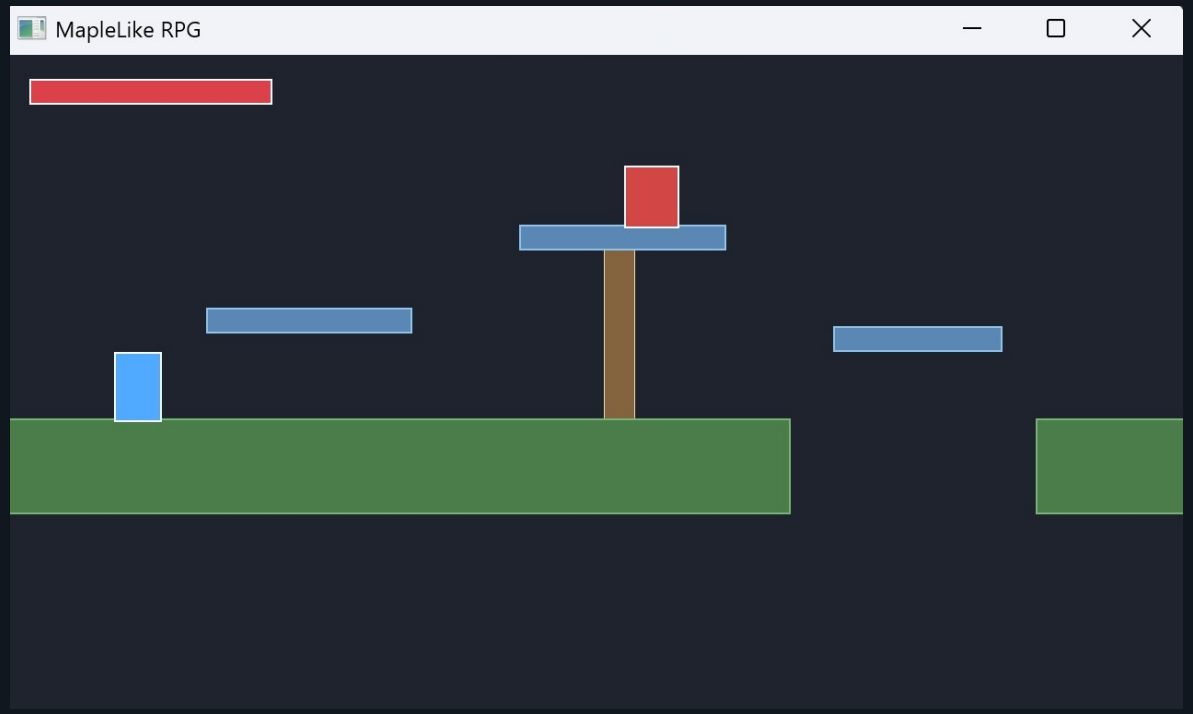
팀전 모드: 로비 버튼 → 방 설정 → 시작 설정 → 인게임으로 값을 넘깁니다(19쪽 문서).

근거 GameEndImageOverlay.cs 310줄은 모두 본인 작성입니다(2026-11-07 "feat: gameEndPanel"). 팀전 모드는 c7348ba(2026-11-03)입니다.

GameEndImageOverlay.cs / HK-INDIE-GO >

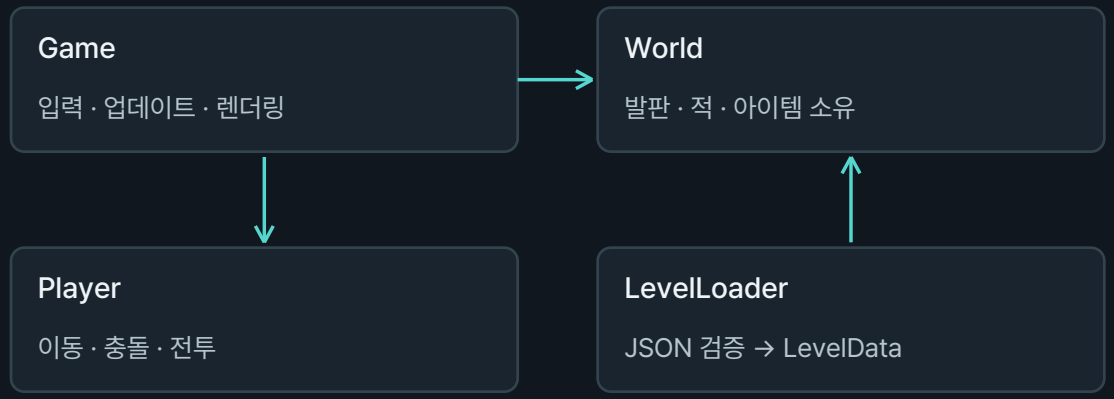
MapleLikeClient · 엔진 없이 만든 게임 루프

2026.07~08 | 개인 프로젝트 | C++ · SFML · JSON · Visual Studio | 2D 액션



시작한 이유

Unity가 대신 처리하던 게임 루프, 물리·충돌, 상태 전환, 데이터 로딩, 렌더링을 직접 만들어 보며 엔진 안쪽의 동작을 이해하려고 시작했습니다.



근거 Game이 객체를 소유하고 갱신 순서를 조율합니다. 경과 시간은 최대 1/60초로 제한합니다. 모든 코드는 본인 작성입니다.

MapleLikeClient / GitHub >

벽처럼 막히던 발판을 단방향 발판으로

첫 버전에서 모든 발판이 막힌 지형처럼 동작하던 문제를, 발판 종류를 나누고 이전 프레임 위치로 착지를 판정해 고쳤습니다.

상황 첫 버전은 모든 발판을 막힌 지형으로 처리했습니다. 공중의 얇은 발판 아래에서 점프하면 머리가 부딪혀 올라가지 못하고, 옆에서 걸어가면 벽처럼 막혔습니다.

원인 위로 이동하다 겹치면 발판 아래로 밀어내고 수직 속도를 0으로 만드는 처리가 발판 종류 구분 없이 적용됐습니다. 수평 충돌도 마찬가지였습니다.

해결 발판을 Solid·OneWay로 나누고 레벨 JSON의 얇은 발판 7개를 oneWay로 바꿨습니다. 단방향 발판은 수평 충돌에서 빼고 이전 프레임의 발 위치와 하강 여부로만 착지시킵니다.

결과 아래에서 뚫고 올라가 위에 착지하고, 아래+점프로 내려갈 수 있습니다. 피격·등반·리스폰 때 단방향 접지 상태도 함께 초기화합니다.

첫 버전 bdebe40: 모든 발판을 같이 처리

```
338  if (velocity.y > 0.f)
     ...
344  else if (velocity.y < 0.f)
346      body.setPosition({ body.getPosition().x,
             platformPosition.y + platformSize.y });
```

수정 8d23f2d: 위에서 내려올 때만 착지

```
491  if (platform.getType() == PlatformType::OneWay)
493      const bool landedFromAbove = velocity.y >= 0.f
494          && previousBottom <= platformPosition.y + 1.f;
495      if (dropThroughTimer > 0.f || !landedFromAbove)
497          continue;
```

근거 두 커밋(bdebe40 → 8d23f2d, 같은 날 2026-08-27)의 코드 차이로 확인한 내용입니다. 오동작은 직접 플레이하며 겪었습니다.

bdebe40 첫 버전 > 8d23f2d 단방향 발판 >

조작감을 위한 입력 보정과 카메라

입력 시점과 발판을 떠난 시간을 잠시 보존해, 사람이 느끼는 타이밍 오차를 받아 주도록 했습니다.

점프 버퍼	착지 직전에 누른 점프를 잠시 보관했다가, 착지하는 순간 버퍼가 남아 있으면 점프합니다.
코요테 타임	발판을 막 떠난 직후에도 점프를 허용합니다. 지면에서 갱신한 타이머가 공중에서 줄어듭니다.
가변 점프	점프 키를 일찍 놓으면 상승 속도를 줄여 낮게 뜁니다.
경계 조건	점프하면 두 타이머를 0으로 만들고 사용 횟수를 올립니다. 걷다 떨어지면 공중 점프 수를 제한하고, 사다리에 오르면 모두 초기화합니다.
카메라	중심을 월드 범위 안으로 제한하고 1 - exp(-속도 × 경과 시간)으로 부드럽게 따라갑니다. 창 비율이 바뀌면 뷰포트로 비율을 유지합니다.

근거 점프 보정·사다리·카메라·HUD 좌표 처리는 Player.cpp, CameraController.cpp에 있습니다. 모두 본인 작성입니다.

Player.cpp / MapleLikeClient >

코드를 고치지 않고 바꾸는 레벨 데이터

JSON 레벨을 검증한 뒤 월드 객체를 만들고, 데이터의 소유와 조회를 나눠 배치를 바꿔도 게임 로직은 그대로 두게 했습니다.

JSON 파일 → LevelLoader 검증 → LevelData → World 객체 생성

좌표 · 크기	숫자 두 개의 배열이며 크기는 양수
발판 타입	solid 또는 oneWay
등반 타입	ladder 또는 rope
오류	필드 이름이 들어간 예외로 알림, 파일 열기 실패는 따로 구분

소유와 조회

World가 발판·적·아이템을 vector로 소유하고, Player에는 span으로 복사 없이 조회 범위만 넘깁니다. vector가 재할당되면 기존 뷰가 무효가 된다는 점도 함께 관리했습니다.

아이템과 화면 좌표

월드에 놓인 ItemDrop과 플레이어가 가진 Inventory를 나누고, 월드 좌표와 화면에 고정된 HUD 좌표를 따로 다뤘습니다.

근거 레벨 검증과 월드 생성은 LevelLoader.cpp, World.cpp에 있습니다. 모두 본인 작성입니다.

LevelLoader.cpp / MapleLikeClient >

Wheel of Fortune · 24시간 게임잼

2026.08 · 약 24시간 | 교내 게임 개발 동아리 · 4인 팀(기획 2 · 개발 1 · 디자인 1) · 단독 프로그래머 | Unity · C# | 로그라이크 턴제



맵 화면 (기존 캡처)

판단	모든 기능과 연출을 똑같이 다듬기 어렵다고 보고, 맵 → 이벤트·전투 → 보상·성장 → 다음 진행의 핵심 루프부터 완성했습니다
구현	이벤트 노드 그래프(인접 리스트, 연속 제한, 가중치 추첨), 룰렛 10칸, 전투·보상·상점·여관, 씬 전환 뒤 진행 상태 복원
배운 점	구현 범위가 넓어질수록 검증에 쓸 시간이 줄어든다는 것을 겪고, 역할 분담과 범위 조정이 왜 중요한지 느꼈습니다

08.18 14:46 첫 커밋 → 08.19 07:54 Demo MVP (약 17시간) → 12:22 마지막 작업 (약 21.5시간) → Windows 빌드 동아리 배포

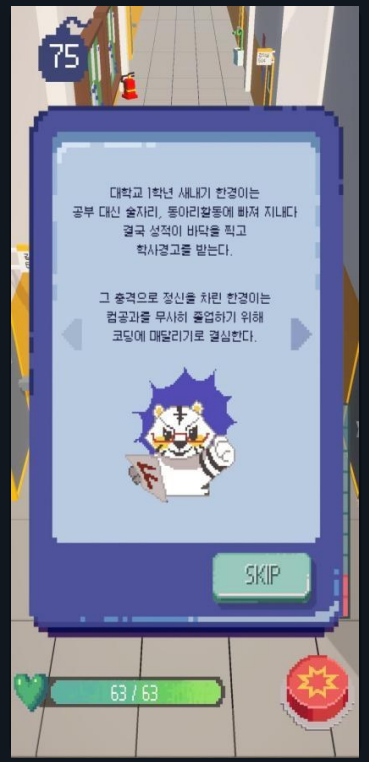
근거 노드·맵 생성·룰렛·진행·세션·전투 스크립트는 모든 줄이 본인 작성이고, 노드 점선 표시와 맵 스크롤은 다른 팀원 코드입니다.

Wheel-of-Fortune / GitHub >

컴공과 키우기 · 보스 전투와 축제 시연

2025.08~11 (이후 2026.02까지 기획 요청 반영) | 교내 게임 개발 동아리 · 7인 팀(기획 1 · 개발 4 · 디자인 2) · PM 겸 프로그래머 | Unity · C# · Android





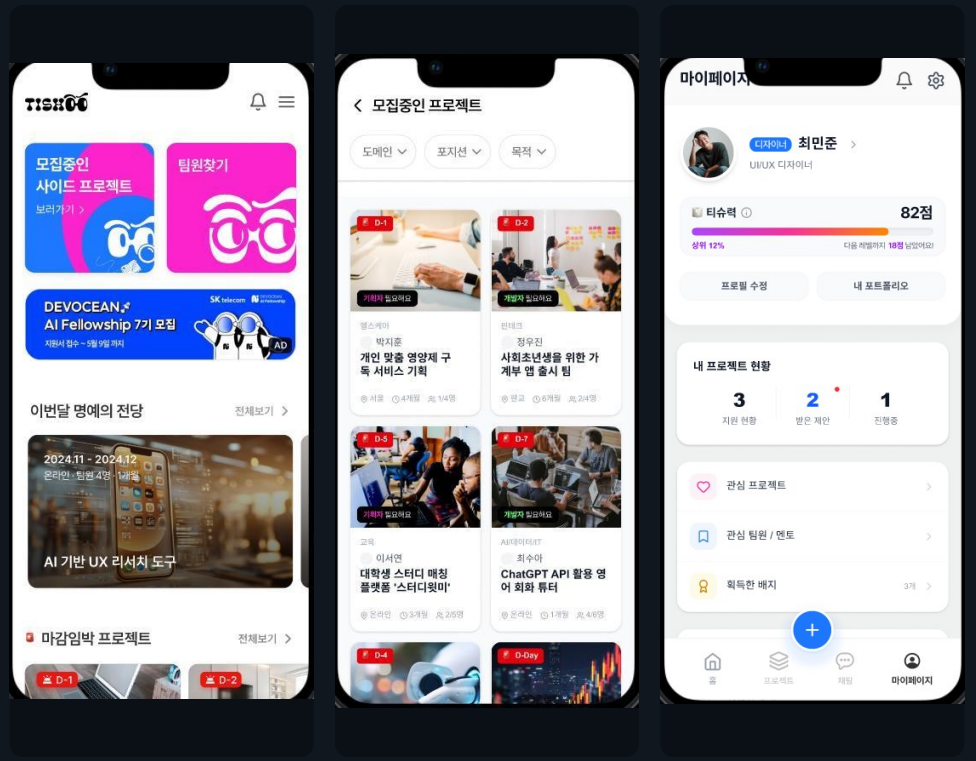
보스	공통 BossBase 위에 Stage 1~5 보스를 나눴습니다(Stage 2~5 본인 커밋). 직선·대각선·부채꼴 공격과 성장·가속 탄을 만들고, 성장·가속은 컴포넌트로 분리해 여러 패턴에서 다시 썼습니다
PM	개발 일정과 기능별 진행을 관리하고, 기능별 브랜치로 작업 영역을 나눠 개발 브랜치에 합쳤습니다. 기능 간 연결을 보며 우선순위를 조정했습니다
결과	Android APK로 빌드해 학교 축제에서 방문객에게 시연했습니다
이후	공식 기간 뒤에도 기획 담당의 플레이 피드백을 받아 보스 체력·탄 피해, 스테이지 타이머(140초 → 90초) 등을 고쳤습니다

근거 저장소 커밋 149개 중 본인 커밋이 59개로 팀에서 가장 많습니다. 페이즈 값은 ScriptableObject로 두고 경과 시간에 맞춰 적용합니다.

MiniGame_2025_2 / dev 브랜치 >

Tishoo · 혼자 맡은 웹 서비스 개발

2025.08~11 | 기업 연계 수업 과제 · 7인 팀(기획 2 · 개발 1 · 디자인 4) · 단독 개발자 | PHP · JavaScript · MySQL/MariaDB | 게임 외 경험



화면	디자인 결과물을 실제 화면으로 옮겨 프로젝트 탐색·생성, 프로필, 마이페이지, 동료 후기 페이지를 만들고 공통 레이아웃을 분리했습니다
데이터	요구사항을 분석해 사용자·프로젝트·참여 정보·동료 후기 테이블을 설계하고, 입력 → PHP → DB → 조회 흐름을 직접 구현했습니다
배포	학교가 제공한 클라우드 서버를 직접 관리했습니다. 문제가 생기면 화면의 JavaScript부터 PHP, DB, 서버 환경까지 따라가며 원인을 찾았습니다
역할	기획·디자인 요구사항을 구현 가능한 기능과 데이터 구조로 바꾸는 일을 맡았습니다

배운 점 개발자가 한 명이라 하나의 기능을 화면·서버·데이터 흐름 전체로 생각해야 했습니다.

sideproject / GitHub >

SKILLS

기술 · 학습 · 연락처

프로젝트에서 실제로 쓴 기술과, 따로 공부한 내용입니다.

Unity · C#	플레이어 행동·전투 판정, 적·보스 AI, 장비·스탯, UI·입력, 씬 전환·저장, WebGL·Android 빌드
C++ · SFML	게임 루프, 이동·충돌, 입력 보정, 카메라, JSON 레벨 검증, vector·span 소유 구분
플랫폼	앱인토스 WebGL 적용, Unity → JavaScript 진동 연결, 출시 뒤 업데이트
협업	Git·GitHub 기능 브랜치와 PR, 공통 정의 설계, 작업 경계 문서, PM
보조	PHP · JavaScript · MySQL/MariaDB, 클라우드 서버 배포

자료구조 선택 기준

중복 대상 판정은 HashSet(평균 O(1)), 환경 객체는 vector로 소유하고 span으로 조회, 맵 연결은 인접 리스트(O(V+E) 공간)로 표현했습니다.

교육 · 자격

C·C++ 프로그래밍(2023), 파이썬 프로그래밍(2023), 머신러닝·딥러닝 과정(2024), 정보처리기사 실기 과정(2026). 정보처리기사 취득 예정(2026.12).

CONTACT

jinu217@naver.com

github.com/jinu217 > jinu217.github.io >

학력 한경국립대학교 소프트웨어융합전공 2027.02 졸업예정 · 병역 산업기능요원 복무 마침(2022.04~2024.03)

공격 ID로 반복 피격 구분 · 사다리와 로프

여러 프레임 이어지는 공격을 한 번만 맞게 하고, 등반 상태를 일반 이동과 나눠 처리했습니다.

공격 ID

새 공격이 시작될 때마다 공격 ID를 올리고, 적은 마지막으로 처리한 ID를 기록합니다. 공격 판정이 여러 프레임 유지돼도 같은 공격은 한 번만 맞습니다. 여러 공격자로 넓히려면 식별 키와 기록 범위를 다시 설계해야 한다는 한계도 정리했습니다.

사다리 · 로프

위아래 입력이 있고 점프 입력이 없을 때 닿은 등반 객체를 찾아 진입합니다. 등반 중에는 수평 위치를 객체 중심에 맞추고 수직 속도만 적용합니다. 점프·좌우 입력이나 위아래 끝에서 상태와 참조를 해제합니다.

Enemy::tryTakeHit

```
46  if (isDead() || attackId == lastHitAttackId ||
        !overlaps(attackBounds))
    ...
51  lastHitAttackId = attackId;
52  --health;
```

Player::updateClimbing

```
278 bool Player::updateClimbing(const std::span<const Climbable>
        climbables)
    ...
293 activeClimbable = findClimbable(climbables);
    ...
296 climbing = true;
    ...
313 activeClimbable = nullptr;
```

근거 MapleLikeClient 8d23f2d 기준입니다. 모든 코드는 본인 작성입니다.

MapleLikeClient / GitHub >

부채꼴 탄막의 각도 계산

플레이어 방향을 기준으로 직선 연사와 부채꼴 발사를 조합하고, 커지거나 작아지는 탄을 컴포넌트로 다시 썼습니다.

부채꼴 계산

탄이 2개 이상이면 전체 각도를 (탄 수 - 1)로 나눈 간격으로, -전체 각도/2에서 시작합니다. 플레이어를 향한 기준 회전에 탄마다 회전을 더해 전방 벡터로 쏩니다.

패턴 조합

혼합 페이즈에서는 발사 횟수를 세어 일정 주기마다 부채꼴을, 나머지는 직선 연사를 쏩니다. Stage 5는 작아지는 탄과 커지는 탄을 번갈아 쏘는 4연발과 부채꼴 패턴을 쏩니다.

FireFanAtPlayer

```
54  void FireFanAtPlayer(int count, float totalAngle)
    ...
60  float step  = count > 1 ? totalAngle / (count - 1) : 0f;
61  float start = -totalAngle * 0.5f;
    ...
65  float yaw = start + step * i;
66  Quaternion rot = Quaternion.Euler(0f, yaw, 0f) * baseRot;
67  SpawnBullet(rot * Vector3.forward);
```

근거 Stage 2~5 보스 스크립트는 본인 커밋입니다(stage2 2025-10-16 ~ boss5 2025-11-14).

MiniGame_2025_2 / dev 브랜치 >

이벤트 맵 그래프 · 룰렛 · 진행 복원

24시간 안에 만든 맵·룰렛·진행 구조를 정리했습니다. 자료구조 선택의 비용도 함께 따졌습니다.

맵 그래프

노드마다 좌표·이벤트 종류·다음 노드 목록(인접 리스트)을 둡니다. 연결할 때 중복과 같은 종류의 연속 횟수를 확인합니다. 상점·여관은 연속을 막고 전투·랜덤은 연속 2회 미만만 허용합니다.

인접 리스트	V개 노드·E개 간선을 O(V+E) 공간으로
중복 검사	List.Contains O(d), 연결이 많아지면 HashSet 검토
인접 행렬	조회 O(1)이지만 $O(V^2)$ 공간

룰렛 · 진행 복원

10칸에 효과를 배치하고 칸을 균등하게 뽑아, 같은 효과의 칸 수가 곧 확률이 됩니다. 결과 칸을 먼저 고르고 목표 각도를 계산해 돌립니다. 완료한 이벤트와 현재 위치는 세션에 보관해 전투 뒤 맵으로 돌아오면 복원합니다.

InGameEventNode.ConnectTo

```
95  if (nextNode != null && nextNode != this &&
        !nextNodes.Contains(nextNode)
96      && CanConnectTo(nextNode.EventType))
    ...
98  nextNodes.Add(nextNode);
99  int nextStreak = eventType == nextNode.EventType ?
        ConsecutiveTypeCount + 1 : 1;
```

근거 노드·룰렛·진행 스크립트는 모든 줄이 본인 작성입니다. 노드 점선 표시는 다른 팀원 코드입니다.

Wheel-of-Fortune / GitHub >

오디오 채널 규칙 · 후기 평점 집계

설정 값을 실제 출력에 반영하는 규칙과, 입력된 후기를 프로필 지표로 바꾸는 계산입니다.

Indie Go · 오디오 설정

BGM	전체·BGM 음소거면 0, 아니면 전체 × BGM
클릭	전체·클릭 음소거면 0, 아니면 전체 × 클릭
토글	전체 음소거면 0, 아니면 전체

채널별 음량 적용

```
213  bgmAudioSource.volume = IsMasterMuted || IsBgmMuted ? 0f :
         MasterVolume * BgmVolume;
214  clickAudioSource.volume = IsMasterMuted || IsClickMuted ? 0f :
         MasterVolume * ClickVolume;
215  toggleAudioSource.volume = IsMasterMuted ? 0f : MasterVolume;
```

Tishoo · 동료 후기 집계

평균 평점	평점 합계 / 후기 수
긍정 비율	4점 이상 후기 수 × 100 / 후기 수
후기 없음	0으로 처리해 나눗셈을 막음
점수	5점 만점 평균 × 20 → 100점

09_view.php · 집계

```
93  $avgRating     = $reviewCount > 0 ? $sumRating / $reviewCount : 0;
    ...
95  $positiveRate  = $reviewCount > 0 ? round($positiveCount * 100 /
        $reviewCount) : 0;
```

근거 음량 값은 0~1로 제한하고 PlayerPrefs에 저장합니다. 후기는 매개변수를 바인딩해 저장합니다. 두 코드 모두 본인 작성입니다.

HK-INDIE-GO > sideproject >